

Sincronización de partidos a Firestore

Spec de implementación para pasarle directo a Claude Code. Repos:

ezecastiglione18/volley_stats_app (frontend, donde vive casi todo el cambio) y

ezecastiglione18/volley_stats_app_backend (sólo la regla de Firestore nueva).

Instrucción para Claude Code: implementar los cambios de la sección 4 tal cual están especificados (son diffs concretos sobre archivos existentes + un archivo nuevo). Las decisiones de diseño de la sección 2 ya están cerradas — no hace falta replantearlas ni pedir confirmación por ellas. Ante cualquier ambigüedad no cubierta acá, priorizar el estilo y los patrones ya presentes en el repo (comentarios en español explicando el *porqué*, tolerancia a fallas de red igual que `SubscriptionController.refresh()`, nunca bloquear la UI esperando red).

1. Contexto y objetivo

Hoy los partidos (equipos, formación, log de jugadas punto a punto) viven sólo en Hive, local a cada dispositivo — es la fuente de verdad para la carga en vivo y el archivo. El objetivo de este cambio es que el **archivo de partidos sea por cuenta** (no por dispositivo): un partido cargado en una tablet debe aparecer también en el archivo del celular de la misma cuenta, sin que eso rompa nada de lo que ya funciona offline.

La carga punto a punto (`MatchController`) y la lectura del archivo/scouting (`match_archive_screen.dart`, `stats_engine.dart`, `scouting_engine.dart`) **no se tocan**: siguen leyendo de Hive exactamente igual que hoy, 100% offline. Lo único que se agrega es una sincronización en segundo plano entre ese Hive local y un espejo en Firestore, para que el archivo termine reflejando todos los partidos de la cuenta, vengan de donde vengan.

2. Decisiones ya tomadas (cerradas)

- El tope de 3 partidos de la versión free cuenta **por cuenta**, no por dispositivo — un usuario free no puede esquivarlo jugando en varios aparatos.
- **No** hace falta partir el log de jugadas en una subcolección aparte — no se espera más granularidad que la actual, y el tamaño de un partido completo está lejos del límite de 1 MiB por documento de Firestore.
- La fecha se guarda como `Timestamp` nativo de Firestore en el espejo (aunque en Hive se siga guardando como `String ISO-8601`, por compatibilidad con lo ya existente).
- El cambio es 100% del lado del cliente (repo frontend) más una regla de seguridad nueva (repo backend). **No** se toca ninguna Cloud Function ni el flujo de códigos promocionales.
- La escritura hacia Firestore es **fire-and-forget**: nunca se espera la red desde la UI. Se apoya en que el SDK de Firestore encola solo los escritos hechos sin conectividad (persisten en disco) y los sincroniza al reconectar — no se implementa una cola de reintentos propia.

3. Modelo de datos en Firestore

```
matches/{uid}/user_matches/{matchId}  
// El documento es VolleyMatch.toJson() con un solo ajuste: el campo  
// "date" pasa de String ISO-8601 a Timestamp nativo (ver  
// MatchSyncService._toFirestoreJson más abajo). El resto de los campos  
// (tournament, ownRoster, sets con sus events/substitutions/sanctions,  
// etc.) viaja tal cual sale de toJson(), sin cambios.
```

uid es el mismo uid de Firebase Auth que ya identifica a la cuenta en account_devices/{uid}.

matchId es el mismo VolleyMatch.id que ya genera generateId('match_') en el cliente — sirve tal cual como id del documento para poder detectar duplicados por igualdad simple.

4. Cambios de archivo por archivo

4.1 — repo backend: firestore.rules

Agregar al mismo archivo que ya tiene las reglas de account_devices y promo_codes (dentro del mismo bloque match /databases/{database}/documents):

```
// Espejo por cuenta de los partidos terminados (ver MatchSyncService en  
// volley_stats_app) — permite que el archivo de partidos sea por CUENTA y  
// no por dispositivo. A diferencia de promo_codes, acá el acceso es  
// directo desde el cliente: no hay cupo ni lógica compartida que proteger,  
// sólo pertenencia por uid, igual que account_devices.  
match /matches/{uid}/user_matches/{matchId} {  
  allow read, write: if request.auth != null && request.auth.uid == uid;  
}
```

Desplegar con `firebase deploy --only firestore:rules` desde el repo backend. Es independiente del build de la app — no requiere una versión nueva de RallyStats para entrar en vigencia.

4.2 — repo frontend: archivo nuevo lib/services/match_sync_service.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';  
  
import '../models/volley_match.dart';  
import '../state/app_data_controller.dart';  
import 'auth_service.dart';  
  
/// Sincroniza los partidos terminados con un espejo en Firestore  
/// (matches/{uid}/user_matches/{matchId}), para que el archivo de  
/// partidos sea por CUENTA y no por dispositivo, sin que la carga en vivo  
/// ni la lectura del archivo local dependan de la red en ningún momento.  
///  
/// Subir (pushFinishedMatch): se llama desde AppDataController.saveMatch  
/// cuando match.status == MatchStatus.finished. Es fire-and-forget: si no  
/// hay conectividad, el propio SDK de Firestore encola el escrito en su  
/// cache local (persiste en disco, sobrevive un reinicio de la app) y lo  
/// sincroniza solo apenas vuelve la conexión. No hace falta una cola de  
/// reintentos propia.  
///  
/// Bajar (pullMissingMatches): se llama al arrancar la app y cada vez que  
/// vuelve a primer plano (mismos lugares donde ya se revalida la  
/// suscripción y el dispositivo, ver main.dart). Trae cualquier partido  
/// que ya exista en Firestore para esta cuenta y todavía no este en el  
/// Hive de este dispositivo, reutilizando el mismo camino que ya usa  
/// "Importar partido" (AppDataController.saveMatch).  
class MatchSyncService {  
  static final MatchSyncService instance = MatchSyncService._();  
  MatchSyncService._();  
  
  final _db = FirebaseFirestore.instance;
```

```

CollectionReference<Map<String, dynamic>> _userMatches(String uid) =>
    _db.collection('matches').doc(uid).collection('user_matches');

/// toJson() guarda "date" como String ISO-8601 (formato de Hive); en el
/// espejo de Firestore conviene guardarlo como Timestamp nativo, por si
/// mas adelante hace falta ordenar/filtrar por fecha del lado del
/// servidor.
Map<String, dynamic> _toFirestoreJson(VolleyMatch match) {
    final json = match.toJson();
    json['date'] = Timestamp.fromDate(match.date);
    return json;
}

/// Inverso de _toFirestoreJson: vuelve a dejar "date" como String
/// ISO-8601 antes de pasarlo a VolleyMatch.fromJson (que espera el
/// formato de Hive, sin tocar el modelo).
Map<String, dynamic> _fromFirestoreJson(Map<String, dynamic> json) {
    final copy = Map<String, dynamic>.from(json);
    final date = copy['date'];
    if (date is Timestamp) {
        copy['date'] = date.toDate().toIso8601String();
    }
    return copy;
}

/// Sube [match] al espejo de la cuenta logueada. No lanza excepciones: el
/// guardado local ya se hizo antes de llamar a esto, así que un error de
/// red acá nunca debe arriesgar ese guardado ni bloquear a quien llama.
Future<void> pushFinishedMatch(VolleyMatch match) async {
    final uid = AuthService.instance.currentUser?.uid;
    if (uid == null) return;
    try {
        await _userMatches(uid).doc(match.id).set(_toFirestoreJson(match));
    } catch (_) {
        // Sin conectividad: el SDK ya dejo el escrito encolado en su cache
        // local y lo reintenta solo. No hace falta reintentar a mano acá.
    }
}

/// Trae a este dispositivo los partidos de la cuenta que esten en
/// Firestore y todavia no en el Hive local. Ordenado por fecha ascendente
/// a proposito: si una cuenta free esta en el limite, importa primero los
/// partidos mas viejos (ver MatchArchiveLimitException mas abajo).
///
/// Best-effort: si falla (sin red), no hace nada – se reintenta solo en
/// el proximo arranque o vuelta a primer plano (ver main.dart).
Future<void> pullMissingMatches(
    AppDataController appData, {
    required bool isPremium,
}) async {
    final uid = AuthService.instance.currentUser?.uid;
    if (uid == null) return;
    try {
        final snapshot = await _userMatches(uid).orderBy('date').get();
        for (final doc in snapshot.docs) {
            if (appData.matches.any((m) => m.id == doc.id)) continue; // ya lo tengo local
            final match = VolleyMatch.fromJson(_fromFirestoreJson(doc.data()));
            try {
                await appData.saveMatch(match, isPremium: isPremium);
            } on MatchArchiveLimitException {
                // Cuenta free que ya llego al tope de 3 (en este dispositivo o en
                // otro): no se importa este partido de mas. No es un error.
            }
        }
    } catch (_) {

```

```
        // Sin conectividad: se reintenta solo en el proximo arranque/foreground.  
    }  
}  
}
```

4.3 — repo frontend: lib/state/app_data_controller.dart

Agregar el import y enganchar la subida dentro de saveMatch:

```
// agregar al principio del archivo, junto a los demas imports:
import 'dart:async';
import '../services/match_sync_service.dart';

// dentro de saveMatch(), reemplazar:
//
// await StorageService.instance.saveMatch(match);
// if (idx == -1) {
//
// por:
//
// await StorageService.instance.saveMatch(match);
// if (match.status == MatchStatus.finished) {
//   // Fire-and-forget: no se espera la red (ver
//   // MatchSyncService.pushFinishedMatch). El guardado local de arriba ya
//   // se hizo, asi que esto nunca debe bloquear ni arriesgar ese guardado.
//   unawaited(MatchSyncService.instance.pushFinishedMatch(match));
// }
// if (idx == -1) {
//   // (el resto de la funcion sigue exactamente igual)
```

Este es el único punto de enganche que hace falta del lado de la subida: saveMatch es el único método por el que pasa cualquier guardado de partido en toda la app (armar formación, terminar un partido en vivo, importar un JSON, editar desde el archivo) — no hay que tocar MatchController ni live_match_screen.dart para nada.

4.4 — repo frontend: lib/main.dart

Agregar el import y dos llamadas a pullMissingMatches, en los mismos dos lugares donde ya se revalida la suscripción y el dispositivo:

```
// agregar al principio del archivo:
import 'services/match_sync_service.dart';

// en _SubscriptionGateState.initState(), agregar la segunda linea:
@override
void initState() {
  super.initState();
  context.read<SubscriptionController>().refresh();
  MatchSyncService.instance.pullMissingMatches(
    context.read<AppDataController>(),
    isPremium: context.read<SubscriptionController>().isPremium,
  );
}

// en _AuthGateState.didChangeAppLifecycleState, agregar la ultima linea:
@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.resumed) {
    context.read<SubscriptionController>().refresh();
    AuthService.instance.revalidateThisDevice();
    MatchSyncService.instance.pullMissingMatches(
      context.read<AppDataController>(),
      isPremium: context.read<SubscriptionController>().isPremium,
    );
  }
}
```

Ninguna de las dos llamadas se espera (await) desde acá — mismo criterio que ya usa SubscriptionController.refresh() en esos mismos puntos: nunca bloquear el arranque ni el resumen de la app esperando red.

5. Limitación aceptada (no es un bug a resolver)

Como `pullMissingMatches` no se espera, hay una ventana breve entre que se abre la app y que termina de traer los partidos de otros dispositivos. Si en ese instante una cuenta free justo inicia un partido nuevo, el chequeo del tope de 3 en `saveMatch` puede evaluarse contra un archivo todavía incompleto. Es el mismo tipo de consistencia eventual que el proyecto ya acepta hoy para la baja de plan y el límite de dispositivos (ver el comentario de `_evictSelfIfOverLimit` en `auth_service.dart`: "no hay ningún mecanismo instantáneo para esto... es eventual"). No corresponde bloquear el arranque de la app esperando la red para cerrar esta ventana — sería inconsistente con el resto del proyecto y perjudica la experiencia offline por un caso límite de bajísima frecuencia.

6. Cómo probarlo

- **Sync básico:** con una cuenta de prueba, terminar un partido en un dispositivo/emulador con conexión, y confirmar que aparece en el archivo de otro dispositivo/emulador logueado con la misma cuenta después de abrir o volver a primer plano la app ahí.
- **Offline en el momento de terminar:** poner el dispositivo en modo avión, terminar un partido (debe guardarse local al toque, sin demora), volver a activar la conexión, y confirmar que el partido aparece solo en Firestore y en el otro dispositivo sin ninguna acción manual.
- **Tope de 3 cross-device:** con una cuenta free de prueba, crear partidos hasta completar 3 entre dos dispositivos distintos, y confirmar que el cuarto partido nuevo se bloquea con `MatchArchiveLimitException` sin importar en cuál de los dos dispositivos se intenta.
- Para no tocar datos reales mientras se prueba, usar el **Firestore Emulator Suite** (Firestore) — ya se usa del lado del backend para probar `redeemPromoCode` — o, si se prueba contra el proyecto real, una cuenta de prueba descartable.

7. Despliegue y quién interviene en cada parte

La regla de Firestore (sección 4.1) se despliega sola, independiente del build de la app — necesita que la máquina esté logueada en la CLI de Firebase (`firebase login`, interactivo, se hace una sola vez). El código del cliente (4.2 a 4.4) sí necesita una versión nueva de RallyStats — bump de versión, `flutter analyze`, build, y pasar de nuevo por la revisión de Google Play. Cuándo conviene lanzar esa versión (en paralelo a otro release en curso o no) es una decisión de Eze, no algo para resolver en la implementación.

Verificar el sync funcionando de punta a punta entre dos dispositivos reales (no sólo que el código compile) conviene que lo confirme Eze en algún momento antes de sacarlo a producción.